Global Defensive Alliances in Star Graphs

Cheng–Ju Hsu[1] Fu–Hsing Wang[2] Yue–Li Wang[1,3]

[1]Department of Information Management, National Taiwan University of Science and Technology, Taipei, Taiwan, Republic of China.

[2]Department of Information Management, Chinese Culture University Taipei, Taiwan, Republic of China.

[3]Department of Computer Science and Information Engineering National Chi Nan University, Nantou, Taiwan, Republic of China.

Abstract

A defensive alliance in a graph $G = (V, E)$ is a set of vertices $S \subseteq V$ where for each $v \in S$, at least half of the vertices in the closed neighborhood of v are in S. A defensive alliance S is called global if every vertex in $V(G) \setminus S$ is adjacent to at least one member of the defensive alliance S. In this paper, we derive an upper bound to the size of the minimum global defensive alliances in star graphs.

1 Introduction

For an undirected graph G, the vertex set and the edge set of G are denoted by $V(G)$ and $E(G)$ respectively. All graphs considered in this paper are finite, undirected, without loops or multiple edges. For any vertex $v \in V(G)$, the *open neighborhood of v* is the set $N(v) = \{ u \in V(G) \mid (u, v) \in E(G) \}$, while $N[v] = N(v) \bigcup v$ denotes the *closed neighborhood of v*. A non-empty set $S \subseteq V(G)$ of vertices is called a *defensive alliance* (respectively, *strong defensive alliance*) if and only if for every $v \in S, |N[v] \bigcap S| \geq |N[v] \setminus S|$ (respectively, $|N[v] \bigcap S| > |N[v] \setminus S|$). One could say that every vertex in S is defended from possible attack by vertices in $V(G) \setminus S$. An alliance S is called *global* if it forms a dominating set (i.e., a set such that every vertex in $V(G) \setminus S$ is adjacent to at least one vertex in S).

The problem of finding a minimum global defensive alliance is NP-complete for general graphs [2]. Several bounds on different types of alliance numbers were obtained in [8]. The study of alliance as a graph-theoretic concept has recently attracted a great deal of attention due to some interesting applications in a variety of areas, including quantitative analysis of secondary RNA structures [5], national defense [7], and fault-tolerant computing [9]. Besides, defensive alliances are the mathematical model of web communities. Adopting the definition of Web community proposed recently by Flake, Lawrence, and Giles [4], "a Web community is a set of web pages having more hyperlinks (in either direction) to members of the set than to non-members".

In this paper, we consider the problem on a particular interconnection network, namely, star graphs. Star graphs were proposed as an attractive alternative to hypercubes with many nice topological properties [3]. Both star graphs and hypercubes provide attractive interconnection scheme for massively parallel systems. Hence characterizations of these architectures have been widely investigated. Star graphs are vertex and edge symmetric, highly regular, strongly hierarchical, and maximally fault-tolerant for connectivity. Due to the strongly hierarchical structure, a star graph can be defined recursively. Moreover, star graphs have many superior advantages over hypercubes such as smaller degree and diameter. In this paper, we give a lower bound and an upper bound to the size of the minimum global defensive alliance for star graphs.

The remaining part of this paper is organized as follows. Section 2 introduces the basic terminol-

ogy and notations. In Section 3, we first establish an upper bound to the size of the minimum global defensive alliance on a star graph with even dimension by using a constructive proof. Applying this result to a star graph with even dimension, we derive an upper bound to the size of the minimum global defensive alliance on a star graph with odd dimension in Section 4. Finally, concluding remarks are given in the last section.

2 Preliminaries

A set $S \subseteq V(G)$ is a *dominating set* of G if for every vertex $u \in V(G) \setminus S$ there exists a vertex $v \in S$ such that u is adjacent to v. We also say that v *dominates* u and u is *dominated* by v. Let $\gamma(G)$ be the cardinality of the *minimum dominating set*. An *induced subgraph* $< S >$ is the maximal subgraph of G with vertex set S. Let $\deg_G(v)$ denote the degree of vertex v in G. In particular, let $\delta(G)$ denotes the minimum degree of G.

A *permutation* is a sequence of elements in which no element appears more than once. Let $N = \{1, 2, \ldots, n\}$ and $p = [p_1, p_2, \ldots, p_n]$ be a permutation, where $p_i \in N$ for all $1 \leq i \leq n$. The *n-dimensional star graph* (n-star for short), denoted by S_n, is an undirected graph consisting of $n!$ vertices labeled by distinct permutations $[p_1, p_2, \ldots, p_n]$. In particular, an n-star is called an *odd dimensional* (respectively, *even dimensional*) star graph if n is odd (respectively, even). For each vertex $v = [p_1, p_2, \ldots, p_n]$, p_i is called the i-th number of v. We also use to denote the symbol at the i-th position of v, i.e., $\beta_i(v) = p_i$, $i = 1, 2 \ldots, n$. Two vertices are connected by an edge if and only if the label of one vertex can be obtained from the label of the other vertex by swapping the first symbol (conventionally, the leftmost) and the i-th symbol, where $2 \leq i \leq n$. Note that an n-star is an edge- and vertex-symmetric regular graph of degree $n-1$. Thus, each vertex labeled with $[p_1, p_2, \ldots, p_{n-1}, i]$ in S_n has $n-1$ neighbors.

The class of star graphs has highly recursive structure. A *k-dimensional substar*, or *k-substar*, is conveniently represented by $\mathcal{K} = x_1 x_2 \ldots x_n$ such that $x_1 = *, x_i \in N \bigcup \{*\}, 2 \leq i \leq n$, where $*$ means "don't care", and there are exactly k $*$'s in $x_1, x_2, \ldots, x_n$. $\mathcal{K}$ is an induced subgraph of S_n in which all vertices of $\mathcal{K}$ are obtained from S_n by replacing each $*$ by a non- $*$ symbol. For example, an S_n can be partitioned into n disjoint $(n-1)$-substars $*^{n-1}i, i = 1, 2, \ldots, n$, where $*^{n-1}i$ is a sequence of $n-1$ $*$'s. Notice that $*^{n-2}i*$, $i = 1, 2 \ldots, n$ are another $(n-1)$-substars of S_n.

In [6], Haynes et al. showed that the global defensive alliance and the total domination numbers are the same for graphs with minimum degree at least two and maximum degree at most three. So, we assume that $S_n, n \geq 5$ for the remaining text of this paper.

3 Even Dimensional Star Graphs

Considering even dimensional star graphs, we define classes of vertex sets A_i and B_{ij} for $i = 1, 2, \ldots, n, j = 1, 2, \ldots, \frac{n}{2} - 2$ in S_n as follows.

$$A_i = \begin{cases} \{v \in V(S_n) \,|\, \beta_1(v) = i+1 \text{ and } \beta_n(v) = i\} \\ \quad \text{if } i \text{ is odd and,} \\ \{v \in V(S_n) \,|\, \beta_1(v) = i-1 \text{ and } \beta_n(v) = i\} \\ \quad \text{if } i \text{ is even.} \end{cases}$$

$$B_{ij} = \begin{cases} \{v \in V(S_n) \,|\, \beta_{\frac{n+2}{2}+j}(v) = i \text{ and} \\ \quad \beta_n(v) = i+1\} \text{ if } i \text{ is odd and,} \\ \{v \in V(S_n) \,|\, \beta_{\frac{n+2}{2}+j}(v) = i \text{ and} \\ \quad \beta_n(v) = i-1\} \text{ if } i \text{ is even.} \end{cases}$$

Arumugam and Kala noted that a set of vertices labeled with a given symbol at the first position plays a leading role for studying the domination problem on star graphs [1]. They have proposed the following lemma.

Lemma 1 *[1] $\{v \in V(S_n) \,|\, \beta_1(v) = i\}$ for some $i \in \{1, 2, \ldots, n\}$ is a minimum dominating set of S_n.*

Lemma 2 *$\bigcup_{i=1}^{n} A_i$ is a dominating set of S_n for even n.*

Proof. We partition S_n into n $(n-1)$-substars by $*^{n-1}i$, for $i = 1, 2, \ldots, n$. For each of the above $(n-1)$-substars, since A_i contains all vertices of $\beta_1(v) = i+1$ if i is odd and $i-1$ if i is even, by lemma 1, A_i is a dominating set of this $(n-1)$-substar. Thus, the lemma follows.

Q. E. D.

Lemma 3 *$\delta(< \bigcup_{i=1,j=1}^{n,\frac{n}{2}-2}(A_i \bigcup B_{ij}) >) \geq \frac{n}{2} - 1$ for even n and $n \geq 5$.*

Proof. Let $G' =< \bigcup_{i=1,j=1}^{n,\frac{n}{2}-2}(A_i \bigcup B_{ij}) >$ and $v \in V(G')$. There are two cases to be considered.

Case 1: $v \in A_i$.

If i is odd, then $\beta_1(v) = i+1$ and $\beta_n(v) = i$. By definition, v has the neighbor u with $\beta_1(u) = i$ and $\beta_n(u) = i+1$. So, $u \in A_{i+1}$. By exchanging the first symbol and the j-th symbol, for $j = \frac{n}{2}+2, \frac{n}{2}+3, \ldots, n-1$, of v. We also obtain another $\frac{n}{2} - 2$

neighbors that are all in B_{ij}. Thus, $\deg_{G'}(v) = \frac{n}{2} - 1$. When i is even, we have $\beta_1(v) = i - 1$ and $\beta_n(v) = i$. It follows that v has the neighbor u, where $\beta_1(u) = i$ and $\beta_n(u) = i - 1$. Clearly, $u \in A_{i-1}$. By exchanging the first symbol and the j-th symbol, for $j = \frac{n}{2} + 2, \frac{n}{2} + 3, \ldots, n - 1$, of v. We can also obtain another $\frac{n}{2} - 2$ neighbors that are all in B_{ij}. Therefore, $\deg_{G'}(v) = \frac{n}{2} - 1$.

Case 2: $v \in B_{ij}$.

Since B_{ij} contains all vertices that are labeled identically at exactly two different positions (the $(\frac{n}{2} + j + 1)$-th symbol and the last symbol). It can be seen that the subgraph induced by B_{ij} is indeed an $(n-2)$-substar of S_n. Therefore, by definition, every vertex in $< B_{ij} >$ has degree $n-3$, implying $\deg_{G'}(v) \geq \frac{n}{2} - 1$.

Q. E. D.

By the same arguments mentioned in the previous lemma, we have the next result.

Corollary 4 *$\delta(G') \geq \frac{n}{2} - 1$ for odd n and $n \geq 5$.*

Theorem 5 *$\gamma_a(S_n) \leq \frac{n-2}{2n-2} n!$ for even n and $n \geq 5$.*

Proof. Lemmas 2 and 3 and Corollary 4 reveal that $\bigcup_{i=1,j=1}^{n,\frac{n}{2}-2}(A_i \bigcup B_{ij})$ is a global defensive alliance of S_n. We are now at a position to compute the size of $\bigcup_{i=1,j=1}^{n,\frac{n}{2}-2}(A_i \bigcup B_{ij})$ in order to establish an upper bound of $\gamma_a(S_n)$.

We first consider $\bigcup_{i=1}^{n} A_i$. Each $A_i, i = 1, 2, \ldots, n$, is in fact an $(n-2)$-substar of S_n. So, the cardinality of A_i is equal to $(n-2)!$. Moreover, it is clear that $A_i \bigcap A_j = \emptyset$ for $i \neq j$. Thus, we have $|\bigcup_{i=1}^{n} A_i| = n \cdot (n-2)!$. Then, it remains to calculate $|\bigcup_{j=1}^{\frac{n}{2}-2} B_{ij}|$. Let $v \in \bigcup_{j=1}^{\frac{n}{2}-2} B_{ij}$. Each $B_{ij}, i = 1, 2, \ldots, n, j = 1, 2, \ldots, \frac{n}{2} - 2$, is an $(n-2)$-substar of S_n. Then $|B_{ij}| = (n-2)!$. Similarly, any two distinct subsets of $\bigcup_{j=1}^{\frac{n}{2}-2} B_{ij}$ are disjoint. We now need to prove $v \notin \bigcup_{i=1}^{n} A_i$. Two cases to be considered depending on i.

Case 1: i is odd.

It follows that $\beta_n(v) = i+1$. Notice that $i+1$ is even. The only possibility that $\{v\} \bigcap \bigcup_{i=1}^{n} A_i \neq \emptyset$ is $v \in \{v \in V(S_n) \,|\, \beta_1(v) = i \text{ and } \beta_n(v) = i + 1\}$. Thus, $\beta_1(v) = i$. This contradicts the fact that $\beta_{\frac{n}{2}+j+1}(v) = i$.

Case 2: i is even.

Clearly, $\beta_n(v) = i - 1$ is odd. The only possibility that $\{v\} \bigcap \bigcup_{i=1}^{n} A_i \neq \emptyset$ is $v \in \{v \in V(S_n) \,|\, \beta_1(v) = i \text{ and } \beta_n(v) = i - 1\}$. Hence, $\beta_1(v) = i$. This contradicts that $\beta_{\frac{n}{2}+j+1}(v) = i$.

Therefore, $|\bigcup_{j=1}^{\frac{n}{2}-2} B_{ij}| = n \cdot (\frac{n}{2} - 2) \cdot (n-2)!$. We conclude that

$$
\begin{aligned}
&|\bigcup_{i=1,j=1}^{n,\frac{n}{2}-2}(A_i \bigcup B_{ij})| \\
&= n \cdot (n-2)! + n \cdot (\tfrac{n}{2} - 2) \cdot (n-2)! \\
&= \tfrac{n-2}{2n-2} n!
\end{aligned}
$$

is an upper bound to the size of the minimum global defensive alliance of S_n with even dimension.

Q. E. D.

4 Odd Dimensional Star Graphs

Considering odd dimensional star graphs, we define classes of vertex sets A_i and B_{ij} for $i = 1, 2, \ldots, n-1, j = 1, 2, \ldots, \frac{n-3}{2}$ in S_n as follows.

$$
A_i = \begin{cases}
\{v \in V(S_n) \,|\, \beta_1(v) = i + 1 \text{ and } \beta_n(v) = i\} \\
\quad \text{if } i \text{ is odd and,} \\
\{v \in V(S_n) \,|\, \beta_1(v) = i - 1 \text{ and } \beta_n(v) = i\} \\
\quad \text{if } i \text{ is even.}
\end{cases}
$$

$$
B_{ij} = \begin{cases}
\{v \in V(S_n) \,|\, \beta_{\frac{n+1}{2}+j}(v) = i \text{ and} \\
\quad \beta_n(v) = i + 1\} \text{ if } i \text{ is odd and,} \\
\{v \in V(S_n) \,|\, \beta_{\frac{n+1}{2}+j}(v) = i \text{ and} \\
\quad \beta_n(v) = i - 1\} \text{ if } i \text{ is even.}
\end{cases}
$$

For convenience, we also define classes of vertex sets C_i and D_{ij} for $i = 1, 2, \ldots, n-1, j = 1, 2, \ldots, \frac{n-5}{2}$ in the $(n-1)$-substar $*^{n-1}n$ as follows.

$$
C_i = \begin{cases}
\{v \in V(*^{n-1}n) \,|\, \beta_1(v) = i + 1 \text{ and} \\
\quad \beta_{n-1}(v) = i\} \text{ if } i \text{ is odd and,} \\
\{v \in V(*^{n-1}n) \,|\, \beta_1(v) = i - 1 \text{ and} \\
\quad \beta_{n-1}(v) = i\} \text{ if } i \text{ is even.}
\end{cases}
$$

$$
D_{ij} = \begin{cases}
\{v \in V(*^{n-1}n) \,|\, \beta_{\frac{n+1}{2}+j}(v) = i \text{ and} \\
\quad \beta_{n-1}(v) = i + 1\} \text{ if } i \text{ is odd and,} \\
\{v \in V(*^{n-1}n) \,|\, \beta_{\frac{n+1}{2}+j}(v) = i \text{ and} \\
\quad \beta_{n-1}(v) = i - 1\} \text{ if } i \text{ is even.}
\end{cases}
$$

Lemma 6 *$\bigcup_{i=1}^{n-1}(A_i \bigcup C_i)$ is a dominating set of S_n for odd n.*

Proof. We partition S_n into n $(n-1)$-substars named $*^{n-1}i$, for $i = 1, 2, \ldots, n$. We first consider $(n-1)$-substars $*^{n-1}i$ for $i = 1, 2, \ldots, n-1$ of S_n. Since A_i dominates all vertices with $\beta_2(v) = j$ for $j \neq i$, and A_i is a dominating set of $*^{n-1}i$.

For $*^{n-1}n$, We also further partition $*^{n-1}n$ into $n-1$ $(n-2)$-substars named $*^{n-1}in$, for $i = 1, 2, \ldots, n-1$. Then since C_i dominates all vertices with $\beta_2(v) = j$ for $j \neq i, n$, and C_i is a dominating set of $*^{n-1}in$. Thus, we conclude that $\bigcup_{i=1}^{n-1}(A_i \bigcup C_i)$ is a dominating set of S_n for odd n.

Q. E. D.

Lemma 7 *$\delta(< (\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij}))$ $\bigcup (\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij})) >) \geq \frac{n-1}{2}$ for odd n and $n \geq 5$.*

Proof. Let $G' =< (\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij}))$ $\bigcup (\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij})) >$ and $v \in V(G')$. There are four cases to be considered.

Case 1: $v \in A_i$.

If i is odd, then $\beta_1(v) = i+1$ and $\beta_n(v) = i$. By definition, v has the neighbor u with $\beta_1(u) = i$ and $\beta_n(u) = i+1$. So, $u \in A_{i+1}$ implies $\deg_{G'}(v) \geq 1$. By exchanging the first symbol and the j-th symbol, for $j = \frac{n+1}{2}+1, \frac{n+1}{2}+2, \ldots, n-1$, of v. We also obtain another $\frac{n-3}{2}$ neighbors that are all in B_{ij}. Thus, $\deg_{G'}(v) = \frac{n-1}{2}$. When i is even, we have $\beta_1(v) = i-1$ and $\beta_n(v) = i$. It follows that v has the neighbor $u \in A_{i-1}$, where $\beta_1(u) = i$ and $\beta_n(u) = i-1$. Furthermore, another $\frac{n-3}{2}$ neighbors that are all in B_{ij} are obtained by exchanging the first symbol and the j-th symbol, for $j = \frac{n+1}{2}+1, \frac{n+1}{2}+2, \ldots, n-1$, of v. Therefore, $\deg_{G'}(v) = \frac{n-1}{2}$.

Case 2: $v \in B_{ij}$.

Since B_{ij} contains all vertices that are labeled identically at exactly two different positions (the $(\frac{n+1}{2}+j)$-th symbol and the last symbol). It can be seen that the subgraph induced by B_{ij} is indeed an $(n-2)$-substar of S_n. Therefore, by definition, every vertex in $< B_{ij} >$ has degree $\frac{n-3}{2}$, implying $\deg_{G'}(v) \geq \frac{n-1}{2}$.

Case 3: $v \in C_i$.

If i is odd, then $\beta_1(v) = i+1, \beta_{n-1}(v) = i$ and $\beta_n(v) = n$. By definition, v has the neighbor u with $\beta_1(u) = n, \beta_{n-1}(u) = i$ and $\beta_n(u) = i+1$. So, $u \in B_{ij}$ implies $\deg_{G'}(v) \geq 1$. By exchanging the first symbol and the j-th symbol, for $j = \frac{n+1}{2}+1, \frac{n+1}{2}+2, \ldots, n-2$, of v. We also obtained another $\frac{n-5}{2}$ neighbors that are all in D_{ij}. So, $\deg_{G'}(v) \geq 1+\frac{n-5}{2}$. Furthermore, one more neighbor $w \in C_i$ of v labeled with $\beta_1(w) = i$ and $\beta_{n-1}(w) = i+1$ is obtained by exchanging the first symbol with the $(n-1)$-th symbol of v. Thus, $\deg_{G'}(v) = 1+\frac{n-5}{2}+1 = \frac{n-1}{2}$. When i is even, we have $\beta_1(v) = i-1, \beta_{n-1}(v) = i$ and $\beta_n(v) = n$. It follows that v has the neighbor $u \in B_{ij}$, where $\beta_1(u) = n, \beta_{n-1}(v) = i$ and $\beta_n(u) = i-1$. By exchanging the first symbol and the j-th symbol, for $j = \frac{n+1}{2}+1, \frac{n+1}{2}+2, \ldots, n-2$, of v. We can also obtained another $\frac{n-5}{2}$ neighbors that are all in D_{ij}. Furthermore, one more neighbor $w \in C_i$ of v labeled with $\beta_1(w) = i$ and $\beta_{n-1}(w) = i-1$ is obtained by exchanging the first symbol with the $(n-1)$-th symbol of v. Thus, $\deg_{G'}(v) = 1+\frac{n-5}{2}+1 = \frac{n-1}{2}$.

Case 4: $v \in D_{ij}$.

Since D_{ij} contains all vertices that are labeled identically at exactly two different positions (the $(\frac{n+1}{2}+j)$-th symbol and the $(n-1)$-th symbol). It can be seen that the subgraph induced by D_{ij} is indeed an $(n-3)$-substar of S_n. Therefore, by definition, every vertex in $< D_{ij} >$ has $n-4$ neighbors in the $(n-3)$-substar. Moreover, each vertex in D_{ij} has a neighbor $u \in C_i$ labeled with $\beta_1(u) = i$ and $\beta_{n-1}(u) = i+1$ (respectively, $\beta_{n-1}(u) = i-1$) if i is odd (respectively, even). Thus, each vertex in D_{ij} must have at least $(n-3)$ neighbors in G'. This implies that $\deg_{G'}(v) \geq \frac{n-1}{2}$.

Q. E. D.

By the same arguments mentioned in the previous lemma, we have the next result.

Corollary 8 *$\delta(< (\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij}))$ bigcup $(\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij})) >) \geq \frac{n-1}{2}$ for even n and $n \geq 5$.*

Theorem 9 *$\gamma_a(S_n) \leq \frac{n-2}{2n-2}n!$ for odd n and $n \geq 5$.*

Proof. Lemmas 6 and 7 and Corollary 8 reveal that $(\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij})) \bigcup (\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij}))$ is a global defensive alliance of S_n. We are now at a position to compute the size of $(\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij})) \bigcup (\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij}))$ in order to establish an upper bound of $\gamma_a(S_n)$ for odd n.

By definition, each vertex in $\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij})$ has the n-th symbol with label with n, while the n-th symbols of the vertices in $\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij})$ are less than n. Obviously, $(\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij})) \bigcap (\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij})) = \emptyset$. That is,

$$|(\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij})) \bigcup (\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij}))| = |\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij})| + |\bigcup_{i=1,j=1}^{n,\frac{n-5}{2}}(C_i \bigcup D_{ij})|.$$

To calculate $|\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij})|$, we apply the same arguments used in Theorem 5 and we get

$$|\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij})| = (n-1)\cdot(n-2)! + (n-1)\cdot\frac{n-3}{2}\cdot(n-2)! = \frac{n-1}{2}(n-1)!.$$

We now compute $|\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij})|$. We first consider $|\bigcup_{i=1}^{n-1} C_i|$. Each $C_i, i = 1, 2, \ldots, n-1$, is in fact an $(n-3)$-substar of S_n. So, the cardinality of C_i is equal to $(n-3)!$. Moreover, it is clear that $C_i \bigcap C_j = \emptyset$ for $i \neq j$. Thus, we have $|\bigcup_{i=1}^{n-1} C_i| = (n-1)\cdot(n-3)!$. Then, it remains to calculate $|\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}} D_{ij}|$. Let $v \in \bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}} D_{ij}$. Each $D_{ij}, i = 1, 2, \ldots, n, j = 1, 2, \ldots, \frac{n-5}{2}$, is indeed an $(n-3)$-substar of S_n. Then $|D_{ij}| = (n-3)!$. Similarly, any two distinct subsets of $\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}} D_{ij}$ are disjoint. Thus, $|\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}} D_{ij}| = (n-1)\cdot\frac{n-5}{2}\cdot(n-3)!$. We now need to prove $(\bigcup_{i=1}^{n-1} C_i) \bigcap \bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}} D_{ij} = \emptyset$. Let $v \in \bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}} D_{ij}$. Two cases to be considered depending on i.

Case 1: i is odd.

It follows that $\beta_{n-1}(v) = i+1$. Notice that $i+1$ is even. The only possibility that $v \in \bigcap\bigcup_{i=1}^{n-1} C_i$ is $v \in \{v \in V(*^{n-1}n) \,|\, \beta_1(v) = i \text{ and } \beta_{n-1}(v) = i+1\}$. Thus, $\beta_1(v) = i$. This contradicts the fact that $\beta_{\frac{n+1}{2}+j}(v) = i$.

Case 2: i is even.

Clearly, $\beta_{n-1}(v) = i-1$ is odd. The only possibility that $v \in \bigcap\bigcup_{i=1}^{n-1} C_i$ is $v \in \{v \in V(S_n) \,|\, \beta_1(v) = i \text{ and } \beta_{n-1}(v) = i-1\}$. Hence, $\beta_1(v) = i$. This contradicts that $\beta_{\frac{n+1}{2}+j}(v) = i$.

Since $(\bigcup_{i=1}^{n-1} C_i) \bigcap \bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}} D_{ij} = \emptyset$. Then we have

$$\begin{aligned}&|\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij})|\\ &= (n-1)\cdot(n-3)! \;+\; (n-1)\cdot\tfrac{n-5}{2}\cdot(n-3)!\\ &= \tfrac{n-3}{2}\cdot(n-1)\cdot(n-3)!.\end{aligned}$$

Finally, it follows that

$$\begin{aligned}&|(\bigcup_{i=1,j=1}^{n-1,\frac{n-3}{2}}(A_i \bigcup B_{ij})) \bigcup (\bigcup_{i=1,j=1}^{n-1,\frac{n-5}{2}}(C_i \bigcup D_{ij}))|\\ &= \tfrac{n-1}{2}(n-1)! \;+\; \tfrac{n-3}{2}\cdot(n-1)\cdot(n-3)!\\ &= (n-1)\cdot(\tfrac{(n-1)!}{2} + \tfrac{n-3}{2}\cdot(n-3)!).\end{aligned}$$

This is an upper bound to the size of the minimum global defensive alliance of S_n with even dimension.

Q. E. D.

5 Concluding Remarks

In this paper, we establish an upper bound $\frac{n-2}{2n-2}n!$ (respectively, $(n-1)\cdot(\frac{(n-1)!}{2} + \frac{n-3}{2}\cdot(n-3)!)$) to the size of the minimum global defensive alliance on n-dimensional star graphs where n is even (respectively, odd). The global defensive alliance that we showed is in fact minimal. An interesting line of further work might be alliance related problems in other interconnection networks.

References

[1] S. Arumugam and R. Kala, Domination parameters of star graph, Ars Combinatoria 44 (1996) 93-96.

[2] A. Cami, H. Balakrishnan, N. Deo, and R. Dutton. On the complexity of finding optimal global alliances, Journal of Combinatorial Mathematics and Combinatorial Computing 58 (2006).

[3] K. Day and A. Tripathi, A comparative study of topological properties of hypercubes and star graphs, IEEE Transactions on Parallel and Distributed Systems 5 (1994) 31-38.

[4] G. W. Flake, S. Lawrence, and C. L. Giles, Efficient identification of Web communities, In Proceedings of the 6th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD-2000) (2000) 150-160.

[5] T. W. Haynes, D. Knisley, E. Seier, and Y. Zou, A quantitative analysis of secondary RNA structure using domination based parameters on trees, BMC Bioinformatics 7 (2006) 108.

[6] T. W. Haynes, S. T. Hedetniemi, and M. A. Henning, Global defensive alliances in graphs, The Electronic Journal of Combinatorics 10 (2003).

[7] P. Kristiansen, S. M. Hedetniemi, and S. T. Hedetniemi, Alliances in graphs, Journal of Combinatorial Mathematics and Combinatorial Computing, 48 (2004) 157-177.

[8] J. A. Rodrguez and J. M. Sigarreta, Spectral study of alliances in gaphs, Discussiones Mathematicae Graph Theory, In press.

[9] K. H. Shafique and R. D. Dutton, On Satisfactory Partitioning of Graphs, Congressus Numer. 154 (2002), 183-194.

Roman Domination on Graphs of Bounded Treewidth*

Sheng-Lung Peng and Yuan-Hsiang Tsai

Department of Computer Science and Information Engineering
National Dong Hwa University
Hualien 974, Taiwan

Abstract

A roman domination function on a graph $G = (V, E)$ is a function $f : V \to \{0, 1, 2\}$ satisfying that every vertex u with $f(u) = 0$ has a neighbor v such that $f(v) = 2$. The weight of a roman domination function is the value $\sum_{v \in V} f(v)$. The roman domination number of a graph G is the minimum weight of all possible roman domination functions on G. In this paper, we show that the roman domination number for a graph with bounded treewidth can be computed in linear time.

1 Introduction

Let $G = (V, E)$ be a finite, simple, and undirected graph. For a vertex $x \in V$, let $N(x) = \{y \in V \mid (x, y) \in E\}$. A vertex subset $D \subseteq V$ is a *dominating set* of G if every vertex which is not in D has a neighbor in D. The DOMINATION PROBLEM for G is to find a minimum dominating set of G. The cardinality of the minimum dominating set is denoted as $\gamma(G)$.

In [3], a new variety of the domination problem called *roman domination* is introduced. A *Roman domination function* on G is a function $f : V \to \{0, 1, 2\}$ satisfying that every vertex x with $f(x) = 0$ has at least one vertex $y \in N(x)$ such that $f(y) = 2$. By definition, we can partition V into three sets V_0, V_1, and V_2 where $V_i = \{v \mid f(v) = i\}$ for $i = 0, 1, 2$. ROMAN DOMINATION PROBLEM for G is to find a Roman dominating function of G such that $2|V_2| + |V_1|$ is the minimum. The minimum value is called *Roman domination number* of G and is denoted by $\gamma_R(G)$. It has been proved that for any graph G, $\gamma_R(G) = \gamma(G) + k$ for any integer k ($2 \leq k \leq \gamma(G)$) [16]. For more background on Roman domination, we refer to [4, 6, 7, 8, 14, 15].

*This work is supported by NSC 95-2221-E-259-010.

It is known that the Roman domination problem on trees can be solved in linear time and it remains NP-complete when restricted to split graphs, bipartite graphs, and planar graphs [3]. The complexity of the Roman domination problem when restricted to block graphs [9], interval graphs, cographs, graphs with bounded cliquewidth, and AT-free graphs can be solved in polynomial time [13].

2 Preliminary

Some domination-like problems on graphs with bounded treewidth can be solved in linear time, *e.g.*, domination [1], bottleneck domination [11], and power domination [5, 12]. The basic idea for these problems is a dynamic programming approach on a tree-like structure called tree decomposition. In the following, we define and present some terms and concept about this terminology.

Let $G = (V, E)$ be a graph. A *tree decomposition* of G is a pair (T, X) where $T = (I, F)$ is a tree and $X = \{X_i \mid i \in I\}$ is a family of subsets of V, and (T, X) satisfies the following properties.

- $\cup_{i \in I} X_i = V$.
- for every edge $(v, w) \in E$, there is an $i \in I$ with $v \in X_i$ and $w \in X_i$.
- for all $i, j, k \in I$, if j is on the path from i to k in T then $X_i \cap X_k \subseteq X_j$.

The *width* of a tree decomposition $((I, F), \{X_i \mid i \in I\})$ is $max_{i \in I}|X_i| - 1$. The *treewidth* of a graph G is the minimum width over all possible tree decompositions of G [10].

A tree decomposition (T, X) is called a *nice* tree decomposition if T is rooted and satisfies the following conditions.

- Every node of T has at most two children.

- If a node i has two children j and k, then $X_i = X_j = X_k$. In this case, we call i *Join Node*.
- If a node i has one child j, then one of the following holds:
  - *Introduce Node*: $|X_i| = |X_j| + 1$ and $X_i \subset X_j$.
  - *Forget Node*: $|X_i| = |X_j| - 1$ and $X_i \supset X_j$.
- If a node has no child, then it is called *Start Node*.

Consider the graph G depicted in Figure 1 as an example. Figure 2 shows a nice tree decomposition of G.

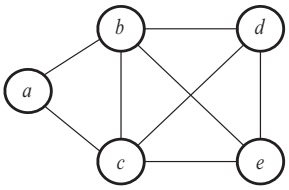

Figure 1: A graph G.

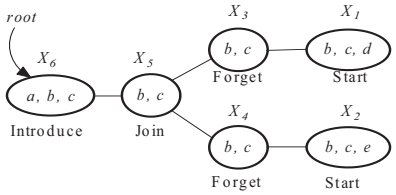

Figure 2: A nice tree decomposition of G.

It was shown in [2] that it can be determined in linear time whether a graph G has treewidth at most k for each constant k. In [10], a nice tree decomposition with treewidth k of G can be constructed given its tree decomposition of treewidth k. In this paper, we propose an algorithm for solving the Roman domination problem on a graph of bounded treewidth given its nice tree decomposition as an input.

3 A linear-time Algorithm

In this section we propose a linear-time algorithm for the Roman domination problem on graphs with bounded treewidth. Let $G = (V, E)$ be a graph with treewidth at most k, and (T, X) is a nice tree decomposition of G. Let T_i denote the subtree of T rooted at node i, X_i denote the vertex subset (or bag) corresponding to node i, and G_i denote the subgraph of G induced by vertices in $\cup_{j \in T_i} X_j$.

For each node i in T, each vertex $x \in X_i$ can be labeled as one number in $\{0^+, 0^-, 1, 2\}$. Number 0^- means that it is not dominated. If one of $N(x)$ is labeled as 2, then x can be labeled as 0^+. Number 1 means that it can be dominated by itself. A labeling is *legal* if it follows the rules of Roman domination or it can be extended to a solution of Roman domination. For example, a labeling with two vertices x and y such that $(x, y) \in E$ and x (respectively, y) is labeled as 2 (respectively, 0^-) is an illegal labeling. For each bag X_i, we create a table to store all the possible labelings for vertices in X_i. During the process, only legal labelings will be referenced. Beside, we also record the weight for each corresponding labeling. Finally, a labeling at the root with the minimum weight is our solution.

We now describe how to obtain a table of legal labelings for a node i from the tables of its children. We consider the four different cases.

1. i **is a start node**: We first list all the possible labelings for all the vertices in X_i. Then we keep all the legal labelings and compute the weight for each legal labeling.
2. i **is an introduce node**: Let j be its child and $X_i = X_j + x$. We add all possible combinations of x for each labeling in X_j. Remove all the illegal labelings and compute the new weight for each legal labeling.
3. i **is a forget node**: Let j be its child and $X_i = X_j - x$. We copy the table from X_j to X_i and do the following.
   (a) Remove these illegal labelings. For example, each labeling with x being labeled as 0^- is not a legal labeling.
   (b) For those legal labelings with the same labels in vertices of $X_j - x$, we keep the one with minimum weight.
4. i **is a join node**: Let j and l be its children. Note that $X_i = X_j = X_l$. We combine the

tables in X_j and X_l into a new table stored in node i by the following steps:

(a) A labeling with a position labeled as 0^+ in node i can be obtained from a labeling with the same position labeled as 0^- (respectively, 0^+ and 0^+) in node j (the remaining positions must have the same labels) combining a labeling with the same position labeled as 0^+ (respectively, 0^- and 0^+) in node l. For example, a labeling $(0^+, \triangle, \square)$ of i is obtained from combining a labeling $(0^-, \triangle, \square)$ of j and a labeling $(0^+, \triangle, \square)$ of l where $\triangle$ and $\square$ are in $\{0^-, 0^+, 1, 2\}$.

(b) For the remaining labelings, we obtain them by combining the same labelings in node j and l.

(c) Compute the new weight for each resulting labeling.

Finally, we find a legal labeling of minimum weight in the root node as our solution.

The following theorem concludes our result.

Theorem 1. *Roman domination problem on graphs with treewidth k can be solved in linear time for every constant $k \geq 1$.*

Proof. The correctness of our algorithm can be obtained easily since we try all possible combinations. In the following we show the time complexity of our algorithm.

1. *Start node*: In this case, it is easy to see that time complexity is bounded by $4^{(k+1)}(k+2)$ since we pick out all possible cases and there are at most $k+1$ vertices in a bag and each vertex has at most 4 labels.

2. *Introduce node*: For the adding vertex, we also choose all possible cases. The time complexity is also bounded by $4^{(k+1)}(k+2)$ for the same reason as Item 1.

3. *Forget node*: It scans all columns in the table for the child and delete columns which is an illegal labeling. The time complexity is also bounded by $4^{(k+1)}(k+2)$ for the same reason as Item 1.

4. *Join node*: It combines two tables stored in the two children of the node. A label 0^+ in a labeling can be obtained from 0^+ and 0^-, 0^- and 0^+, and 0^+ and 0^+. So the computation is at most $\sum_{r=0}^{s}\binom{s}{r}2^{s-r}3^r = 5^s$ for s is the size of the node. Since s is at most $k+1$, the time complexity in this node is $O(5^k)$.

Finally, checking whether a labeling in a bag is legal or not will take $O(k^2)$ time. Since the tree decomposition of G has at most $4n$ nodes [10], the time complexity is bounded by $O(k^2 \times 5^k n)$. □

Consider Figure 2 as an example. In our algorithm, we work from leaves to the root in T. We first consider the bag $X_1 = \{b, c, d\}$, each vertex in X_1 may be labeled as 0^+, 0^-, 1, or 2. So we create a $4^3 \times (3+1)$ table to store the information of labelings for X_1 as follows.

Table 1: Information of the *start* node of X_1.

vertices	labels							
b	0^-	0^-	0^+	...	1	1	2	2
c	0^-	0^+	0^+	...	1	1	1	2
d	0^-	0^-	0^-	...	1	2	2	2
weight	0	0	0	...	3	4	5	6

In the *forget* node $X_3 = \{b, c\}$, we first copy the table from X_1 and then refine it. Finally, we obtain a table as follows.

Table 2: Information of the *forget* node of X_3.

vertices	labels						
b	0^-	0^-	0^+	...	0^+	2	2
c	0^-	0^+	0^+	...	2	1	2
d^*	1	1	1	...	0^+	0^+	0^+
weight	1	1	1	...	2	3	4

* The information of this row will not be used for the next process.

It is easy to see that the information of X_2 (respectively, X_4) is similar to X_1 (respectively, X_3). As it goes to the *join* node X_5, the information is shown in Table 3.

Table 3: Information of the *join* node of X_5.

vertices	labels						
b	0^-	0^+	1	...	0^+	1	2
c	0^-	0^+	0^+	...	2	2	2
weight	2^*	2	2	...	2	3	4

* The label of d in X_1 is 1 and the label of e in X_2 is also 1. Thus the combining weight is 2

Finally, let us see the *introduce* node X_6. The vertex a can be labeled as 0^+, 0^-, 1, or 2. So the table of X_6 is four times the size of the table of X_5. The final table of X_6 is shown in Table 4.

Table 4: Information of the *introduce* node of X_6.

vertices	labels						
a	0^+	0^+	0^+	...	1	1	2
b	2	0^+	2	...	2	2	2
c	0^+	2	1	...	1	2	2
weight	2	2	3	...	4	5	6

Since X_6 is the root, our algorithm will output 2 as the solution. By a backtracking, we can find the corresponding labeling of (a, b, c, d, e) which is $(0, 2, 0, 0, 0)$.

4 Conclusion

In this paper, we provide a linear-time algorithm to solve the Roman domination problem on graphs with bounded treewidth. Whether the Roman domination problem on permutation graphs can be solved in polynomial time is still open.

References

[1] N. Betzler, R. Niedermeier, and J. Uhlmann, Tree decompositions of graphs: saving memory in dynamic programming, *Discrete Optimization* 3 (2006) 220–229.

[2] H. L. Bodlaender, A linear time algorithm for finding tree-decompositions of small treewidth, *SIAM J. Comput.* **25** (1996) 1305–1317.

[3] E. J. Cockayne, P. A. Jr. Dreyer, S. M., Hedetniemi, and S. T. Hedetniemi, Roman domination in graphs, *Discrete Mathmatics* **278** (2004) 11–22.

[4] P. A. Jr. Dreyer, Applications and variations of domination in graphs, Ph.D. Thesis, Rutgers University, The State University of New Jersey, New Brunswick, NJ, 2000.

[5] J. Guo, R. Niedermeier, and D. Raible, Improved algorithms and complexity results for power domination in graphs, *FCT* 2005, LNCS **3623**, pp. 172–184, 2005.

[6] M. A. Henning, A characterization of Roman trees, *Discuss. Math. Graph Theory* **22** (2002) 225–234.

[7] M. A. Henning, Defending the Roman Empire from multiple attacks, *Discrete Mathematics* **271** (2003) 101–115.

[8] M. A. Henning and S. T. Hedetniemi, Defending the Roman EmpireVA new strategy, *Discrete Mathematics* **266** (2003) 239–251.

[9] C.-H. Hsu, C.-S. Liu, and S.-L. Peng, Roman domination on block graphs, *Proceedings of the* 22^{nd} *Workshop on Combinatorial Mathematics and Computation Theory*, 2005, pp. 188–191

[10] T. Kloks, *Treewidth–Computations and Approximatioms*, LNCS **842**, Springer, Berlin, 1994.

[11] T. Kloks, D. Kratsch, C.-M. Lee, and J. Liu, Improved bottleneck domination algorithms, *Discrete Applied Mathematics* **154** (2006) 1578–1592.

[12] J. Kneis, D. Molle, S. Richter, and P. Rossmanith, Parameterized power domination complexity, *Information Processing Letters*, **98** (2006) 145–149.

[13] M. Liedloff, T. Kloks, J. Liu, S.-L. Peng, Roman domination over some graph classes, *WG* 2005, LNCS **3787**, pp. 103–114, 2005.

[14] C. S. ReVelle and K. E. Rosing, Defenders imperium Romanum: A clas problem in military strategy, *Amer. Math. Monthly* **107** (2000) 585–594.

[15] I. Stewart, Defend the Roman Empire, *Scientific American* **281** (1999) 136–139.

[16] H.-M. Xing, X. Chen, and X.-G. Chen, A note on Roman domination in graphs, *Discrete Mathematics* **306** (2006) 3338–3340.